Shaheed Zulfikar Ali Bhutto Institute of Science & Technology

DEPARTEMENT OF ROBOTICS & ARTIFICIAL INTELLIGENCE

Total Marks: 2.5

Obtained Marks: _____

Artificial Neural Network

Lab Task#02

Last date of Submission:14-10-2026

Submitted To: Hassaan Mujtaba

Student Name: Zainab Sohail

Reg. Number: 24108240

DATASET1:Water Potability (CLASSIFICATION)

```
import pandas as pd
import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report

water = pd.read_csv("water_potability.csv")

water.columns = water.columns.str.strip().str.lower()

print(water.shape)
print(water.columns.tolist())
print(water.head())
```

```
(3276, 10)
['ph', 'hardness', 'solids', 'chloramines', 'sulfate', 'conductivity', 'organic_carbon', 'trihalomethanes', 'turbidity', 'potability']
         ph    hardness        solids  chloramines     sulfate  conductivity  \
0       NaN  204.890455  20791.318981     7.300212  368.516441    564.308654
1  3.716080  129.422921  18630.057858     6.635246         NaN    592.885359
2  8.099124  224.236259  19909.541732     9.275884         NaN    418.606213
3  8.316766  214.373394  22018.417441     8.059332  356.886136    363.266516
4  9.092223  181.101509  17978.986339     6.546600  310.135738    398.410813

   organic_carbon  trihalomethanes  turbidity  potability
0       10.379783        86.990970   2.963135           0
1       15.180013        56.329076   4.500656           0
2       16.868637        66.420093   3.055934           0
3       18.436524       100.341674   4.628771           0
4       11.558279        31.997993   4.075075           0
```

```
water = water.dropna(subset=["potability"]).copy()

X_water = water.drop(columns=["potability"])
y_water = water["potability"]

X_water = X_water.fillna(X_water.median())

print("Missing values handled.")
print("Total records:", len(water))
print("Target classes:")
print(y_water.value_counts())
```

```
Missing values handled.
Total records: 3276
Target classes:
potability
0    1998
1    1278
Name: count, dtype: int64
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X_water,
    y_water,
    test_size=0.2,
    random_state=42,
    stratify=y_water
)

water_model = LogisticRegression(max_iter=2000)

water_model.fit(X_train, y_train)

water_predictions = water_model.predict(X_test)

print("Water Potability Classification Results")
print("Accuracy:", round(accuracy_score(y_test, water_predictions), 3))
print(classification_report(y_test, water_predictions))
```

```
Water Potability Classification Results
Accuracy: 0.61
              precision    recall  f1-score   support

           0       0.61      1.00      0.76       400
           1       0.00      0.00      0.00       256

    accuracy                           0.61       656
   macro avg       0.30      0.50      0.38       656
weighted avg       0.37      0.61      0.46       656
```

```
import gradio as gr

def predict_water(*values):
    row = pd.DataFrame([values], columns=X_water.columns)
    row = row.fillna(X_water.median())

    prediction = water_model.predict(row)[0]

    if prediction == 1:
        return "Water is Potable"
    return "Water is Not Potable"

water_app = gr.Interface(
    fn=predict_water,
    inputs=[
        gr.Number(label=column, value=float(X_water[column].median()))
        for column in X_water.columns
    ],
    outputs="text",
    title="Water Potability Classification"
)

water_app.launch()
```

```
* Running on local URL:  http://127.0.0.1:7863
* To create a public link, set `share=True` in `launch()`.
```

```
* Running on local URL:  http://127.0.0.1:7863
* To create a public link, set `share=True` in `launch()`.
```

Water Potability Classification

ph
7.036752103833548

hardness
196.96762686363076

solids
20927.833606520187

chloramines
7.130298973883081

sulfate
333.0735457458888

output

Flag

sulfate
333.0735457458888

conductivity
421.8849682800544

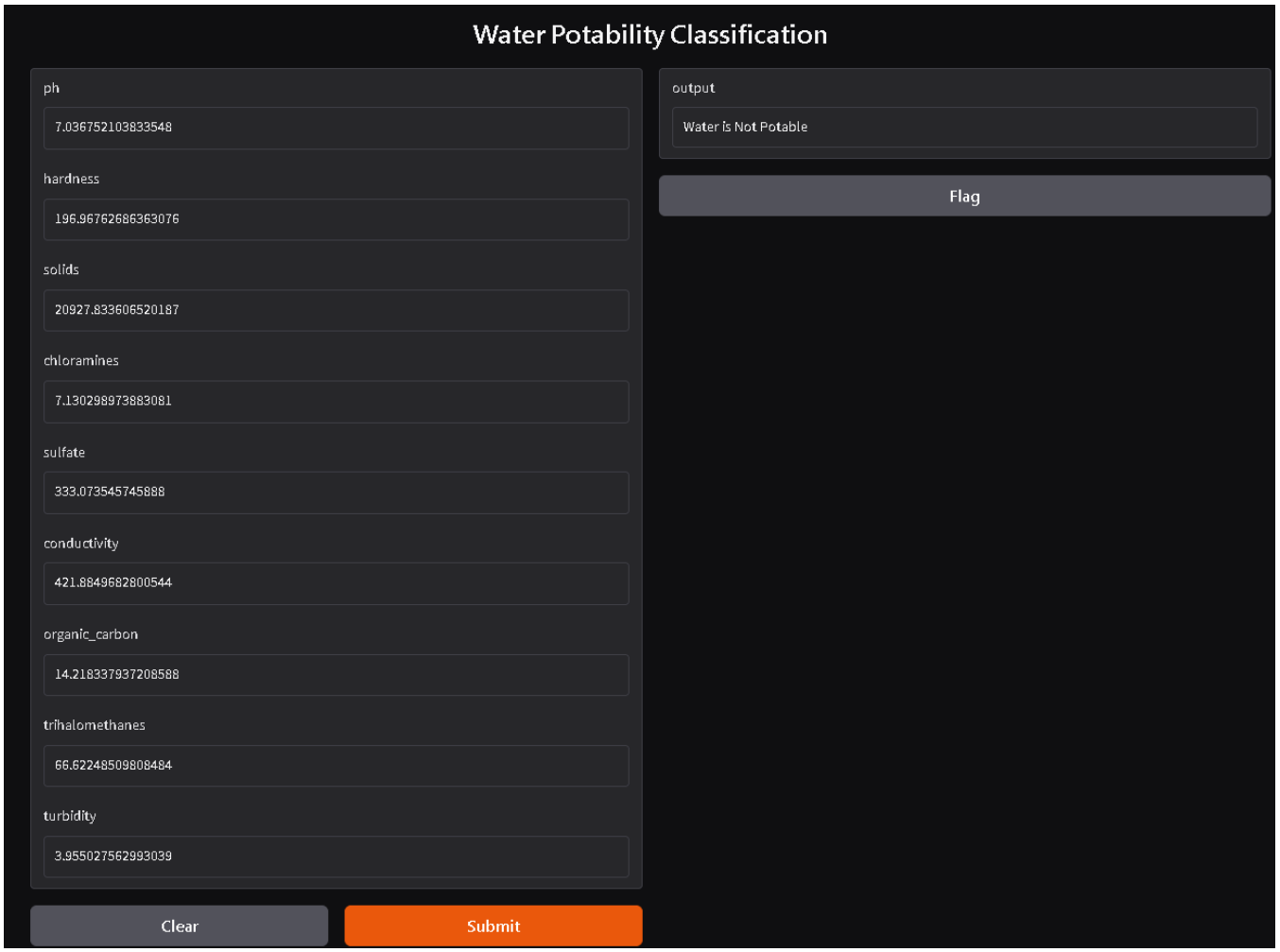
organic_carbon
14.218337937208588

trihalomethanes
66.62248509808484

turbidity
3.955027562993039

Clear

Submit




```python
from flask import Flask, request, render_template_string

water_flask = Flask("water_app")

water_html = """
<h2>Water Potability Classification</h2>
<form method="post">
{% for column in columns %}
  {{ column }}:
  <input type="number" step="any" name="{{ column }}"
         value="{{ defaults[column] }}" required><br><br>
{% endfor %}
  <button type="submit">Predict</button>
</form>
<h3>{{ result }}</h3>
"""

@water_flask.route("/", methods=["GET", "POST"])
def water_home():
    result = ""
```

```python
@water_flask.route("/", methods=["GET", "POST"])
def water_home():
    result = ""

    if request.method == "POST":
        values = [
            float(request.form[column])
            for column in X_water.columns
        ]
        result = predict_water(*values)

    return render_template_string(
        water_html,
        columns=X_water.columns,
        defaults=X_water.median().to_dict(),
        result=result
    )

water_flask.run(port=5002, debug=False, use_reloader=False)
```

```
 * Serving Flask app 'water_app'
 * Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead
 * Running on http://127.0.0.1:5002
Press CTRL+C to quit
127.0.0.1 - - [12/Oct/2026 00:49:49] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [12/Oct/2026 00:49:49] "GET /favicon.ico HTTP/1.1" 404 -
127.0.0.1 - - [12/Oct/2026 00:49:51] "POST / HTTP/1.1" 200 -
```



Water Potability Classification

ph: 7.036752103833548

hardness: 196.96762686363076

solids: 20927.833606520187

chloramines: 7.130298973883081

sulfate: 333.073545745888

conductivity: 421.8849682800544

organic_carbon: 14.218337937208588

trihalomethanes: 66.62248509808484

turbidity: 3.955027562993039

Predict

Water is Not Potable

DATASET 2: Mushrooms dataset

```
#--------DATASET 2----------------------------------
mushrooms = pd.read_csv("mushrooms.csv")

mushrooms.columns = mushrooms.columns.str.strip().str.lower()

print(mushrooms.shape)
print(mushrooms.columns.tolist())
print(mushrooms.head())
```

```
(8124, 23)
['class', 'cap-shape', 'cap-surface', 'cap-color', 'bruises', 'odor', 'gill-attachment', 'gill-spacing', 'gill-size', 'gill-color', 'stalk-shape', 'stalk-r
oot', 'stalk-surface-above-ring', 'stalk-surface-below-ring', 'stalk-color-above-ring', 'stalk-color-below-ring', 'veil-type', 'veil-color', 'ring-number',
'ring-type', 'spore-print-color', 'population', 'habitat']
  class cap-shape cap-surface cap-color bruises odor gill-attachment  \
0     p         x           s         n       t    p               f
1     e         x           s         y       t    a               f
2     e         b           s         w       t    l               f
3     p         x           y         w       t    p               f
4     e         x           s         g       f    n               f

  gill-spacing gill-size gill-color  ... stalk-surface-below-ring  \
0            c         n          k  ...                        s
1            c         b          k  ...                        s
2            c         b          n  ...                        s
3            c         n          n  ...                        s
4            w         b          k  ...                        s

  stalk-color-above-ring stalk-color-below-ring veil-type veil-color  \
0                      w                      w         p          w
1                      w                      w         p          w
2                      w                      w         p          w
3                      w                      w         p          w
4                      w                      w         p          w

  ring-number ring-type spore-print-color population habitat
0           o         p                 k          s       u
1           o         p                 n          n       g
2           o         p                 n          n       m
3           o         p                 k          s       u
4           o         e                 n          a       g

[5 rows x 23 columns]
```

```
mushrooms = mushrooms.dropna(subset=["class"]).copy()

y_mushrooms = mushrooms["class"].map({"e": 0, "p": 1})

X_mushrooms = mushrooms.drop(columns=["class"])
X_mushrooms = X_mushrooms.fillna("missing")

X_mushrooms = pd.get_dummies(X_mushrooms, drop_first=True)

print("Features:", X_mushrooms.shape)
print("Classes: 0 = Edible, 1 = Poisonous")
```

```
Features: (8124, 95)
Classes: 0 = Edible, 1 = Poisonous
```

```
X_train_m, X_test_m, y_train_m, y_test_m = train_test_split(
    X_mushrooms,
    y_mushrooms,
    test_size=0.2,
    random_state=42,
    stratify=y_mushrooms
)

mushroom_model = LogisticRegression(max_iter=2000)

mushroom_model.fit(X_train_m, y_train_m)

mushroom_predictions = mushroom_model.predict(X_test_m)

print("Mushroom Classification Results")
print("Accuracy:", round(accuracy_score(y_test_m, mushroom_predictions), 3))
print(classification_report(y_test_m, mushroom_predictions))
```

```
Mushroom Classification Results
Accuracy: 0.999
              precision    recall  f1-score   support

           0       1.00      1.00      1.00       842
           1       1.00      1.00      1.00       783

    accuracy                           1.00      1625
   macro avg       1.00      1.00      1.00      1625
weighted avg       1.00      1.00      1.00      1625
```

```
def predict_mushroom(*values):
    row = pd.DataFrame([values], columns=mushrooms.drop(
        columns=["class"]
    ).columns)

    row = pd.get_dummies(row)
    row = row.reindex(columns=X_mushrooms.columns, fill_value=0)

    prediction = mushroom_model.predict(row)[0]

    if prediction == 1:
        return "Classification: Poisonous"
    return "Classification: Edible"

mushroom_inputs = [
    gr.Dropdown(
        choices=sorted(mushrooms[column].dropna().astype(str).unique().tolist()),
        value=sorted(mushrooms[column].dropna().astype(str).unique().tolist())[0],
        label=column
    )
    for column in mushrooms.drop(columns=["class"]).columns
]

mushroom_app = gr.Interface(
    fn=predict_mushroom,
    inputs=mushroom_inputs,
    outputs="text",
    title="Mushroom Classification"
)

mushroom_app.launch()
```

```
* Running on local URL:  http://127.0.0.1:7864
* To create a public link, set `share=True` in `launch()`.
```

```
* Running on local URL:  http://127.0.0.1:7864
* To create a public link, set `share=True` in `launch()`.
```

Mushroom Classification

cap-shape: b

cap-surface: f

cap-color: b

bruises: f

odor: a

output: Classification: Poisonous

Flag

n

ring-type: e

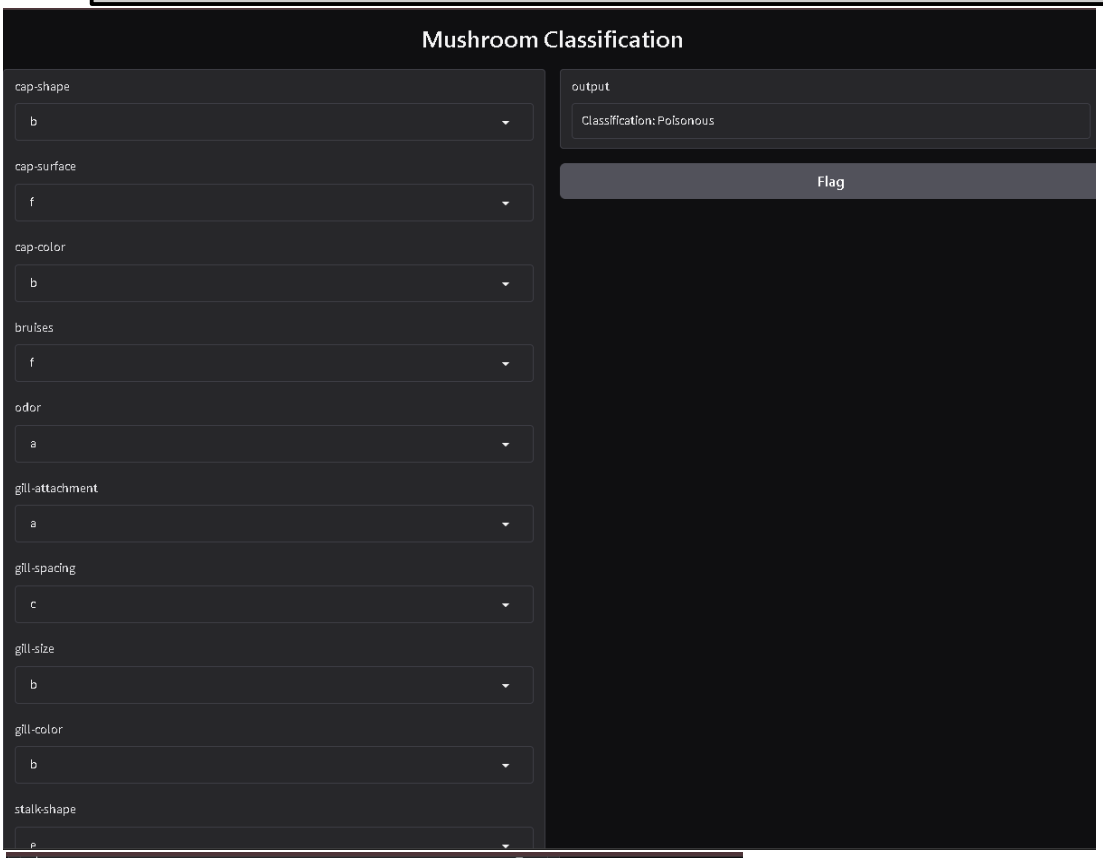
spore-print-color: b

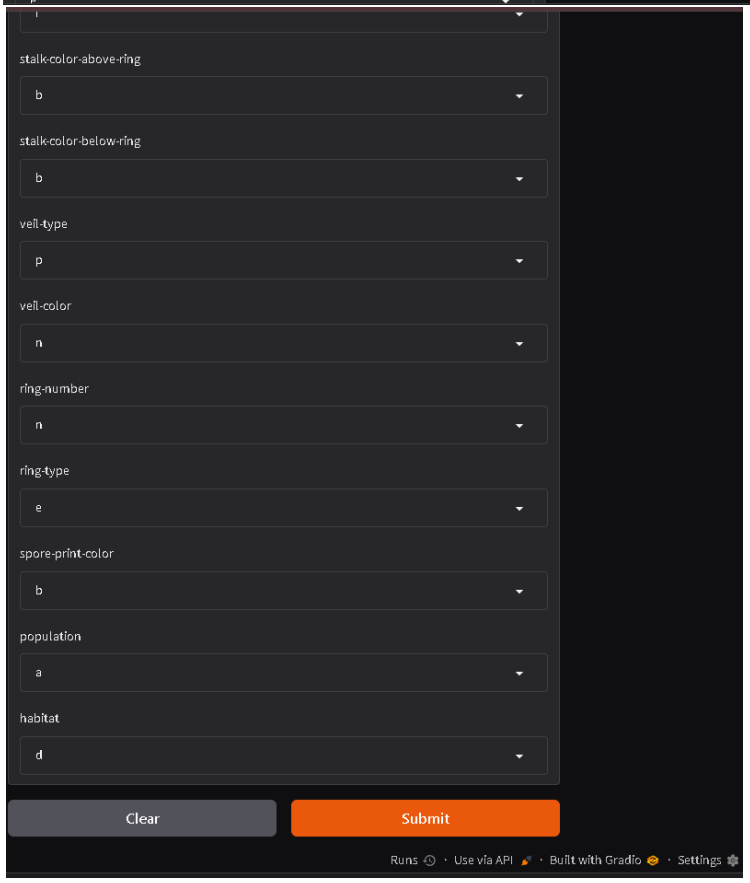
population: a

habitat: d

Clear

Submit


Mushroom Classification
cap-shape
b
cap-surface
f
cap-color
b
bruises
f
odor
a
gill-attachment
a
gill-spacing
c
gill-size
b
gill-color
b
stalk-shape
e
output
Classification: Poisonous
Flag


stalk-color-above-ring
b
stalk-color-below-ring
b
veil-type
p
veil-color
n
ring-number
n
ring-type
e
spore-print-color
b
population
a
habitat
d
Clear
Submit
Runs
Use via API
Built with Gradio
Settings

```
mushroom_flask = Flask("mushroom_app")

mushroom_html = """
<h2>Mushroom Classification</h2>
<form method="post">
{% for column, choices in options.items() %}
  {{ column }}:
  <select name="{{ column }}">
  {% for choice in choices %}
    <option value="{{ choice }}">{{ choice }}</option>
  {% endfor %}
  </select><br><br>
{% endfor %}
  <button type="submit">Predict</button>
</form>
<h3>{{ result }}</h3>
"""

mushroom_options = {
    column: sorted(
        mushrooms[column].dropna().astype(str).unique().tolist()
    )
    for column in mushrooms.drop(columns=["class"]).columns
}

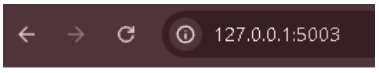
@mushroom_flask.route("/", methods=["GET", "POST"])
def mushroom_home():
    result = ""

    if request.method == "POST":
        values = [
            request.form[column]
            for column in mushrooms.drop(columns=["class"]).columns
        ]
        result = predict_mushroom(*values)

    return render_template_string(
        mushroom_html,
        options=mushroom_options,
        result=result
    )

mushroom_flask.run(port=5003, debug=False, use_reloader=False)
```

```
 * Serving Flask app 'mushroom_app'
 * Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on http://127.0.0.1:5003
Press CTRL+C to quit
127.0.0.1 - - [12/Oct/2026 00:58:30] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [12/Oct/2026 00:58:30] "GET /favicon.ico HTTP/1.1" 404 -
127.0.0.1 - - [12/Oct/2026 00:58:33] "POST / HTTP/1.1" 200 -
```



Mushroom Classification

cap-shape: b

cap-surface: f

cap-color: b

bruises: f

odor: a

gill-attachment: a

gill-spacing: c

gill-size: b

gill-color: b

stalk-shape: e

stalk-root: ?

stalk-surface-above-ring: f

stalk-surface-below-ring: f

stalk-color-above-ring: b

stalk-color-below-ring: b

veil-type: p

veil-color: n

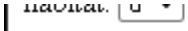
ring-number: n

ring-type: e

spore-print-color: b

population: a

habitat: d



Classification: Poisonous